Vec-LUT: Vector Table Lookup for Parallel Ultra-Low-Bit LLM Inference on Edge Devices

ACM MobiSys 2026, Cambridge, UK (Best Paper Award Runner-Up)

Xiangyu Li[*1] Chengyu Yin[*†2] Weijun Wang[1] Jianyu Wei[3] Ting Cao[‡1] Yunxin Liu[1]
lixiangy22@mails.tsinghua.edu.cn

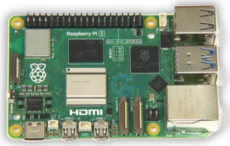
[1]**Institute for AI Industry Research (AIR), Tsinghua University** [2]Beijing Jiaotong University
[3]University of Science and Technology of China

* Equal contribution. Sorted by surname in alphabetical order.
† Work done during internship at AIR, Tsinghua University.
‡ Corresponding author (tingcao@mail.tsinghua.edu.cn).

Scaling Down Bits for On-Device LLM

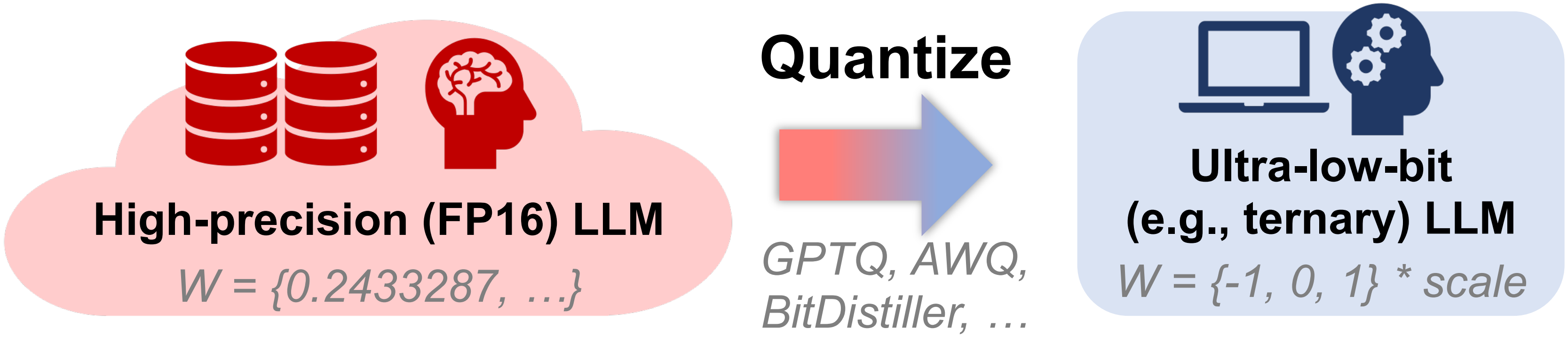
On-device / cloud: a trade-off of intelligence vs. cost / speed /…

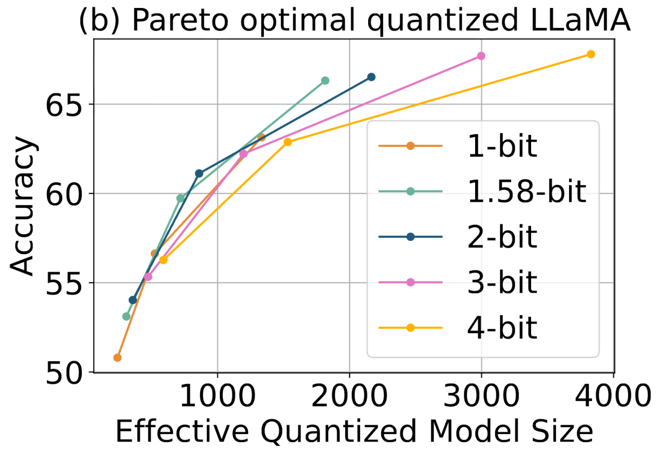
- Cloud AI: slow and expensive; most intelligent for general use.
- On-device AI: fast and cheap; sufficient for down-stream tasks [1].



Low-bit quantization: a *de facto* standard for on-device LLMs.

- ***Ultra-low-bit (esp. ternary)*** **LLMs demonstrate optimal trade-offs** [2, 3].





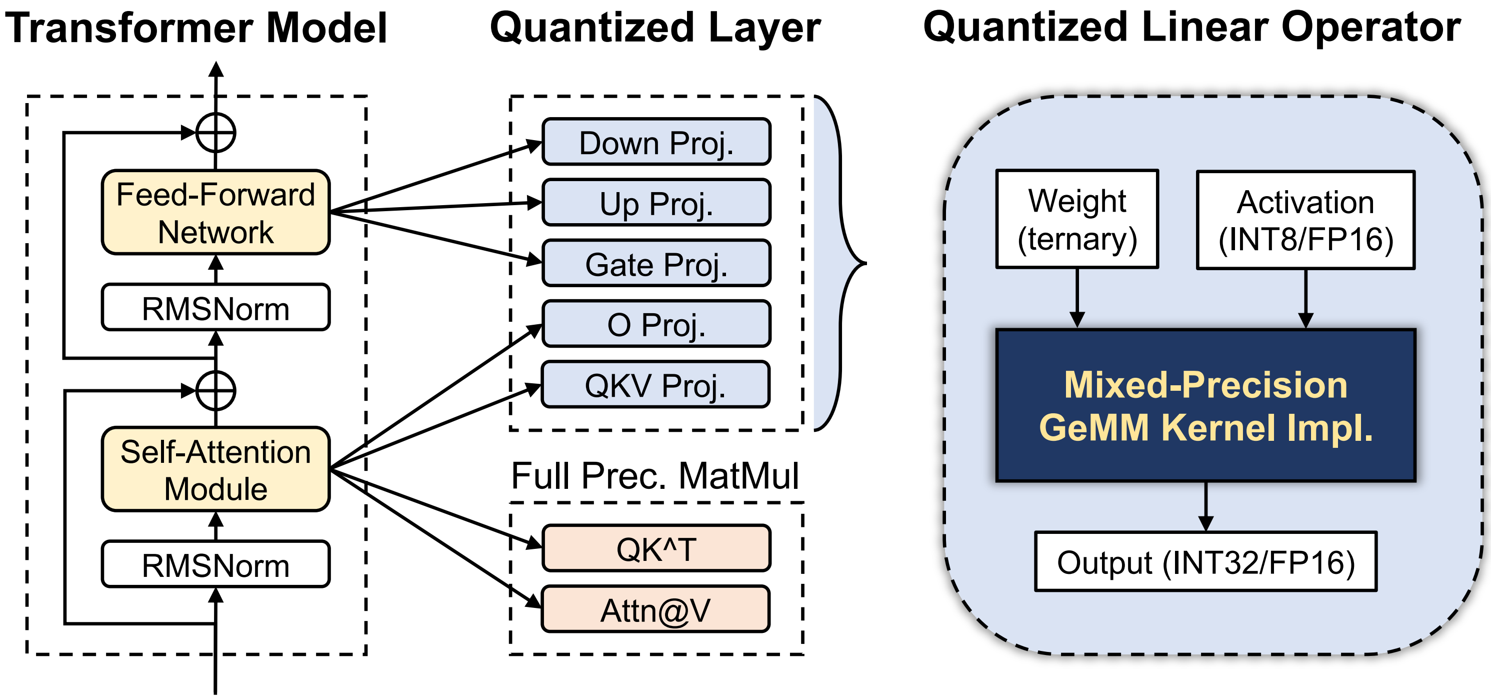
[1] Belcak P., et al. Small Language Models are the Future of Agentic AI." *2025,* ***NVIDIA, Position Paper.***
[2] Liu Z., et al. "ParetoQ: Improving Scaling Laws in Extremely Low-Bit LLM Quantization." ***NeurIPS 2025, Meta AI.***
[3] Huang Z., et al. "TENET: An Efficient Sparsity-Aware LUT-Centric Architecture for Ternary LLM Inference on Edge." *2025,* ***Microsoft.***

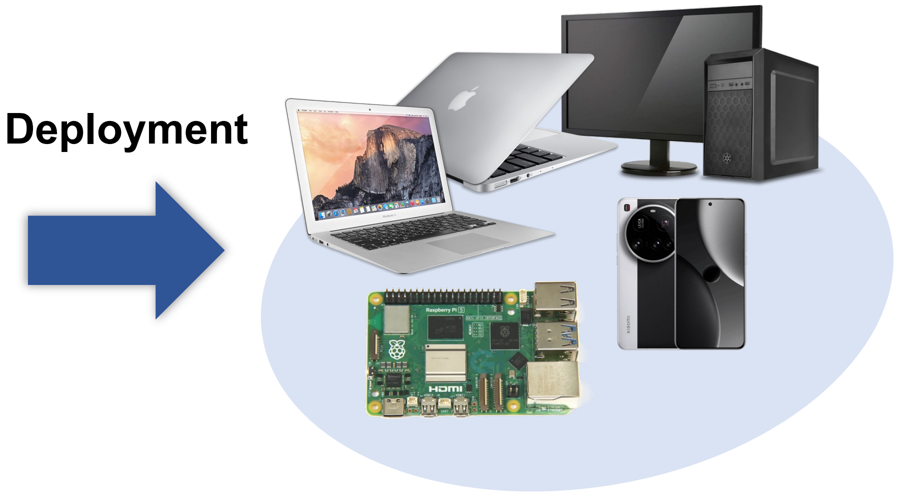
Can Lower Bits Provide Higher Speed?

Not always! Only with proper inference kernel design.

- Quantized LLM inference is dominated by *mixed-precision GeMM (mpGeMM).*
- **Common hardware (CPU/GPU/…) does not natively support mpGeMM.**
    - E.g., 1.58-bit weight multiplying 16-bit activation.

How to efficiently implement
mpGeMM without native support?
—— Lookup Table (LUT)



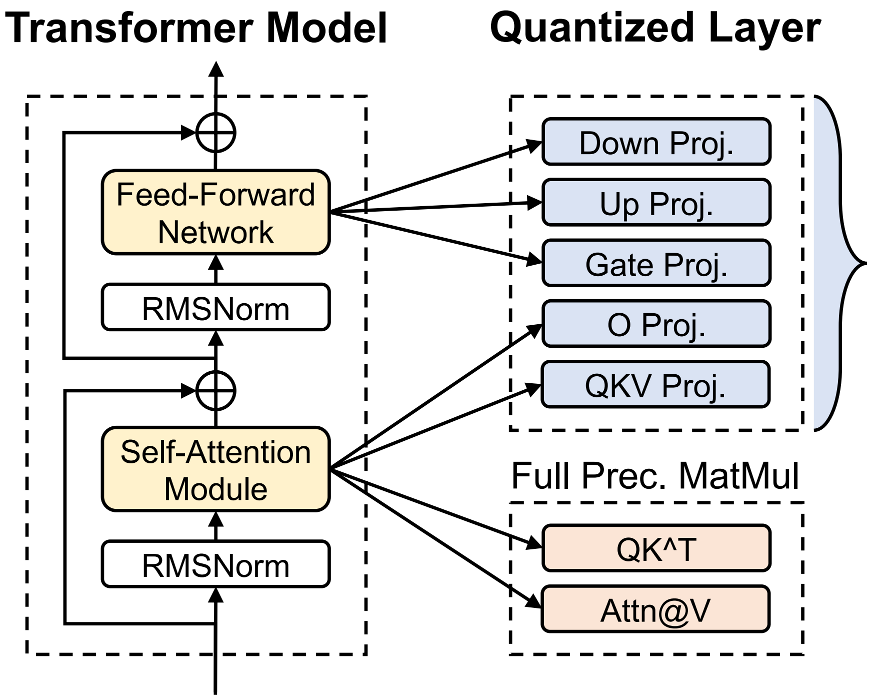


From Dequant. to Lookup Table (LUT)

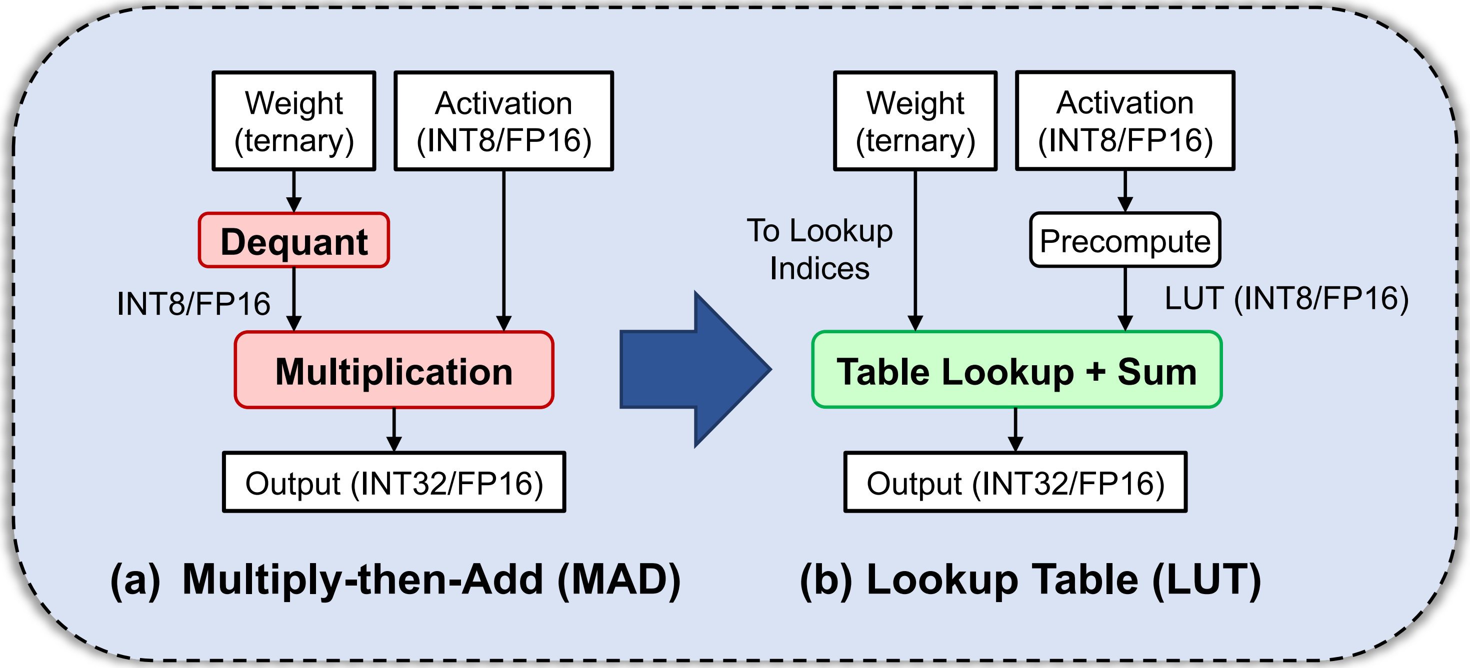
Dequantization and multiplication costs vanish low-bit's gain.

Lookup table (LUT)-based mpGeMM is dequantization-free.

- *This is essential for achieving actual speedup from lower bits.*



MAD (dequant.) vs. LUT-based mpGeMM Kernel Impl.

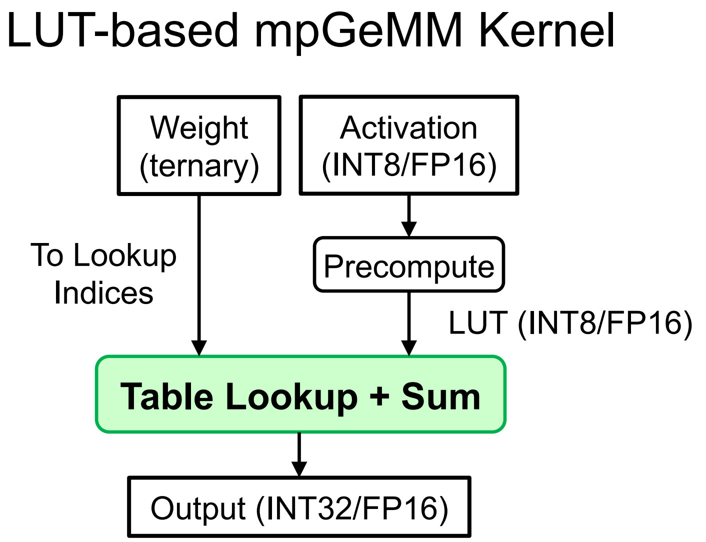


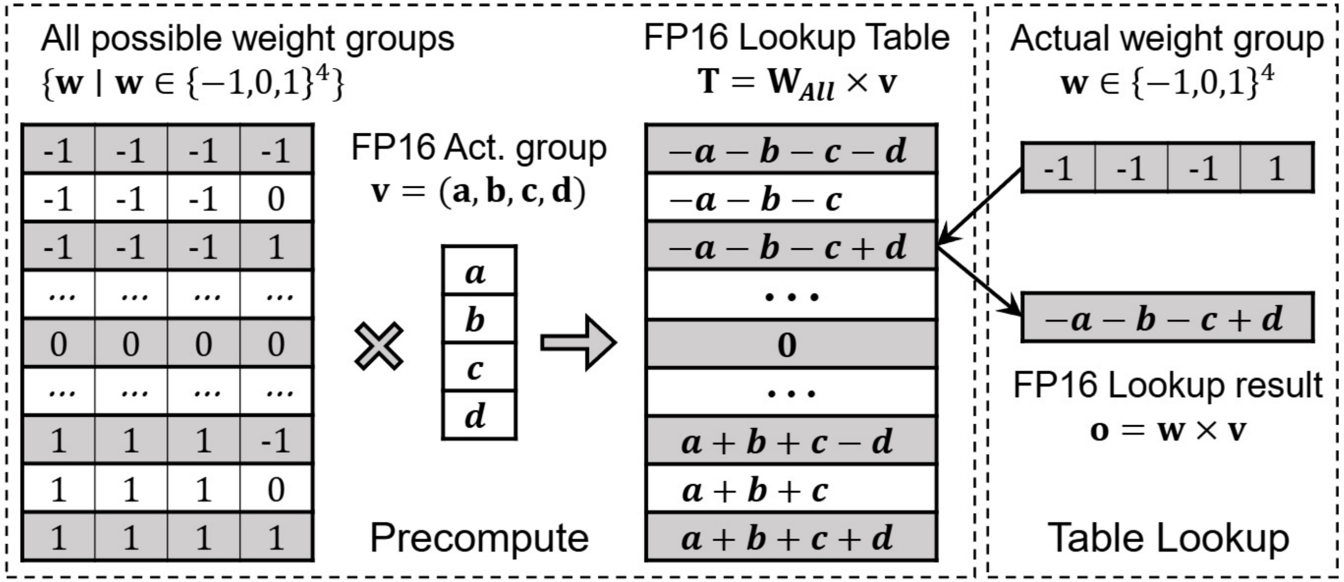
Preliminary: How Does LUT Work?

LUT-based mpGeMM bypasses dequantization by:

1. Enumerating weight patterns (e.g., {-1,0,-1,1}) as *lookup indices.*
2. Precomputing {activation * weight} results as LUT elements.
3. Iterating the actual weight and lookup precomputed results (reused).

Gain: Limited low-bit weight patterns (e.g., 3^4) → LUT reused many times.

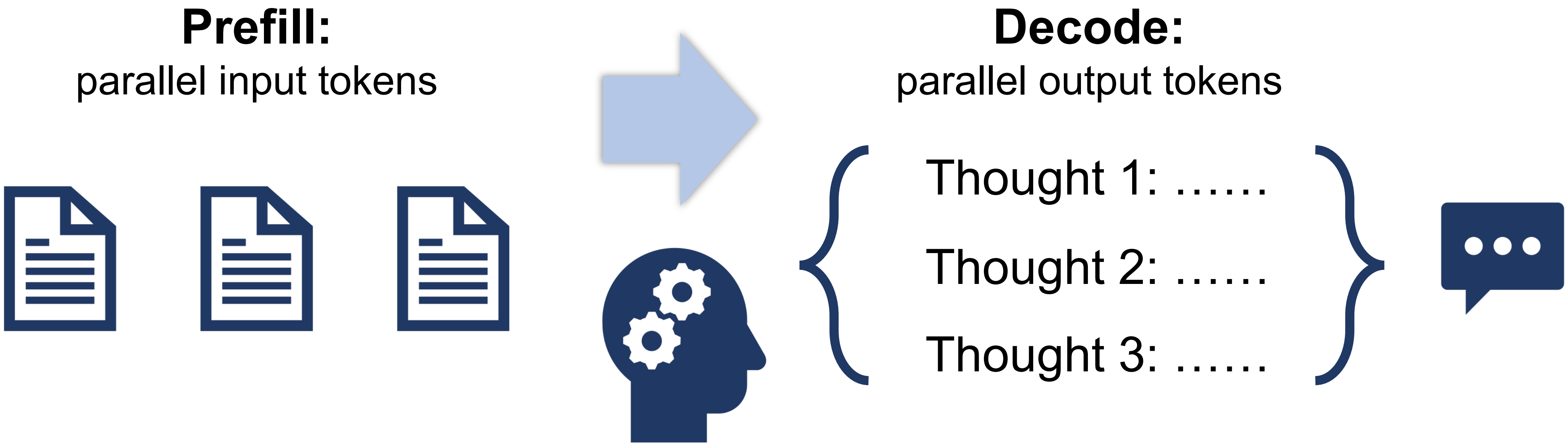




Inefficiency of LUT in Parallel Inference

Parallel inference: an important part of LLM-based applications.

- Definition: multiple tokens processed together instead of one-by-one.
- **Examples: prefill (medium / large context), parallel / speculative decoding.**



Inefficiency of LUT in Parallel Inference

Parallel inference: an important part of LLM-based applications.

- Definition: multiple tokens processed together instead of one-by-one.
- **Examples: prefill (medium / large context), parallel / speculative decoding.**

Existing LUT-based kernels behave poorly in such scenarios.

- Existing: memory bandwidth util. <40% (even slower than non-LUT kernels).
- **Root cause: the repetitive 1→1 lookup (scalar LUT) for multiple tokens.**

Missing opportunity: index sharing for multiple-token lookups.

- The indices, converted from weights, are **naturally shareable** by tokens.
- **We propose vector LUT to utilize this.**

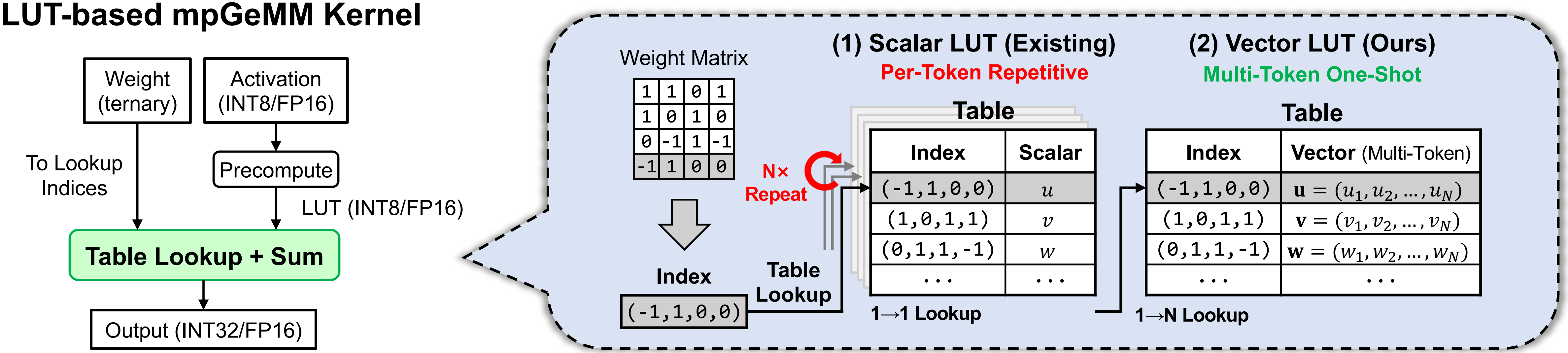
From Scalar LUT to Vector LUT

Vector LUT: 1→N lookup with a unified table, sharing one index.

- Main difference: unified table with vector elements, not scalar.
- Primary gain: reducing lookup operations by N×.
- No reliance on hardware lookup instructions, more flexible.

Technical challenges: memory bandwidth and cache capacity.

LUT-based mpGeMM Kernel



Vec-LUT: Challenges and Techniques

How to design tensor layouts for memory bandwidth efficiency?

- All tensor layout should match this paradigm for contiguous memory access.
- Non-contiguous memory access can degrade performance by up to **12×.**

Vector LUT-Centric Tensor Layout

Ensuring contiguous memory access via offline & runtime reorganization.

How to design LUT access to avoid cache thrashing?

- LUT size expanded by **N×** (e.g., 100s of MiB), exceeding typical cache size.
- Table lookup is inherently **random**, which might lead to a poor cache hit rate.

Cache-Aware Streamed Lookup

Streamlines the precompute-lookup process in tiles within cache capacity.

vlut.cpp: End-to-End Implementation

We implement and integrate Vec-LUT into llama.cpp [1].

- Supports a wide range of edge devices with the ubiquitous CPU backend.
- Pure C/C++ with no external dependencies, built easily on any devices.
- **Source code: https://github.com/OpenBitSys/vlut.cpp .**

vlut.cpp is part of OpenBitSys. Other awesome projects (by Dayou):

- BitDistiller (ACL 2024): QAT with self-distillation for ultra-low-bit LLMs.
- BitDecoding (HPCA 2026): LLM decoding with low-bit KV cache.
- **Follow us on GitHub: https://github.com/OpenBitSys .**

[1] https://github.com/ggml-org/llama.cpp

On-Device Evaluation & Takeaways

We evaluate Vec-LUT across 5 devices and 3 models.

- Devices (ARM/x86): PC, laptop, smartphone, SBC (Pi), CPU server (AWS).
- Model parameters (different backbones): 1B, 3B, 8B.

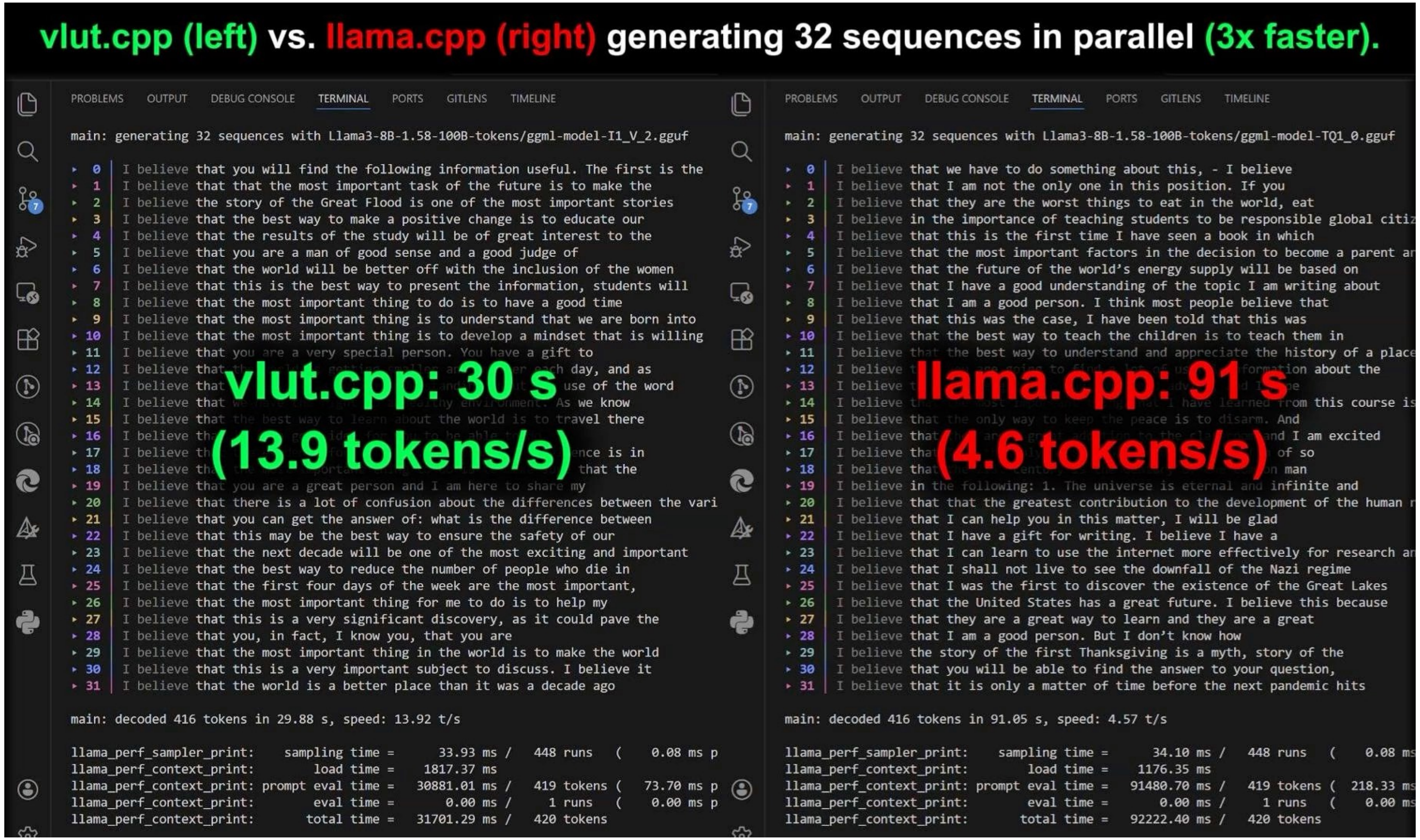
Takeaways:

- **Fast:** up to 4.2× (I1, BPW=1.6) and 2.6× (I2, BPW=2.0) E2E speedup.
- **Compact:** 1.6 bits per weight (BPW) packing w.o. strict shape requirements.
- **Generic:** supports mixed prefilling and decoding (e.g., continuous batching).
- **Energy-efficient:** up to 2.1× reduction of energy cost per token.
- **Competitive to NPU:** 1.1× (2 cores) on Snapdragon 8 Elite vs. llama.cpp.

[1] https://github.com/ggml-org/llama.cpp

Parallel Decoding: 3× E2E Speedup

Configuration: Llama3 8B decoding on a single CPU core.



When to Use Ultra-Low-Bit?

- Domain-specific tasks (not requiring world knowledge).
- Enough training data available (for QAT).
- Memory usage / inference speed is the priority.

Toward Agentic / Physical AI

We are working on:

- **Quantization-aware training and inference systems.**
  - 2-bit training and serving: in collab. with Hisense, releasing soon.
  - More complex tasks (e.g., agentic): WIP.
- **Self-evolving physical agents.** Recent preprints:
  - OxyGen [1]: accelerating multi-task VLA inference on robots.
  - ActProbe [2]: detecting robot policy failures ahead of time.
  - EmbodiSkill [3]: skill reflection and refinement after deployment.
  - MemCompiler [4]: state-conditioned agent memory compilation.

Discussion / collaboration / join us :)

[1] Li, X., et al. “OxyGen: Unified KV Cache Management for VLA Inference under Multi-Task Parallelism.” 2026.
[2] Huang, B., et al. “ActProbe: Action-Space Probe for Early Failure Detection of Generative Robot Policies.” 2026.
[3] Ju, R., et al. “EmbodiSkill: Skill-Aware Reflection for Self-Evolving Embodied Agents.” 2026.
[4] Ding, X., et al. “MemCompiler: Compile, Don't Inject--State-Conditioned Memory for Embodied Agents.” 2026.

Thanks!

Xiangyu Li @AIR, THU

Homepage: https://xxxxyu.github.io

Contact: lixiangy22@mails.tsinghua.edu.cn